SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	T.Praveen Kumar	**Reg. No.:**	192472234
Section / Batch:	B.E-CSE-AI	**Date:**	16-07-2026

Code of Conduct

I T.Praveen Kumar (192472234) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T.Praveen Kumar

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________

Name of the student : T.Praveen Kumar

Register number : 192472234

Course code : DSA007

Q1. Resume Information Extraction

Aim

To extract candidate information from a resume using Regular Expressions and determine eligibility.

Algorithm

1. Start the program.
2. Read the resume text.
3. Extract name, email, mobile, skills and experience using regex.
4. Display extracted details.
5. Check eligibility (minimum 2 years and Python skill).
6. Display eligibility status.
7. Stop.

Code

```
import re
resume = '''
Name: Sumanth
Email: sumanth123@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning
Experience: 3 years
'''
name = re.search(r"Name:\s*(.*)", resume)
email = re.findall(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
mobile = re.findall(r"\b[6-9]\d{9}\b", resume)
skills = re.findall(r"\b(Python|Java|SQL|Machine Learning|NLP)\b", resume)
experience = re.search(r"(\d+)\s*years", resume)
print("Candidate Summary")
print("Name:", name.group(1))
print("Email:", email)
print("Mobile:", mobile)
print("Skills:", skills)
print("Experience:", experience.group(1), "years")
if int(experience.group(1)) >= 2 and "Python" in skills:
print("Status: Eligible")else:
print("Status: Not Eligible")
```

Sample Input

Resume details as shown in the code.

Sample Output

Candidate Summary

Name: Sumanth

Email: ['sumanth123@gmail.com']

Mobile: ['9876543210']

Skills: ['Python', 'Java', 'SQL', 'Machine Learning']

Experience: 3 years

Status: Eligible

Result

The program successfully extracted candidate information and determined eligibility.

Q2. Product Search System

Aim

To perform product searches using Regular Expressions.

Algorithm

8. Start the program.
9. Store product names.
10. Read the keyword.
11. Perform exact, prefix, suffix and partial matching.
12. Display matching products.
13. Display report.
14. Stop.

Code

```
import re
products=["Laptop","Gaming Mouse","Wireless Keyboard","Smart Phone","Phone Cover","Bluetooth
Speaker","Laptop Bag","Mouse Pad"]
keyword=input("Enter keyword: ")
exact=[p for p in products if re.fullmatch(keyword,p,re.IGNORECASE)]
prefix=[p for p in products if re.match(keyword,p,re.IGNORECASE)]
suffix=[p for p in products if re.search(keyword+r"$",p,re.IGNORECASE)]
partial=[p for p in products if re.search(keyword,p,re.IGNORECASE)]
print("Exact Match:",exact)
print("Prefix Match:",prefix)print("Suffix Match:",suffix)
print("Partial Match:",partial)
print("Report")
print("Exact:",len(exact))
print("Prefix:",len(prefix))
print("Suffix:",len(suffix))
print("Partial:",len(partial))
```

Sample Input

Phone

Sample Output

Exact Match: []

Prefix Match: ['Phone Cover']

Suffix Match: ['Smart Phone']

Partial Match: ['Smart Phone', 'Phone Cover']

Report

Exact:0 Prefix:1 Suffix:1 Partial:2

Result

The program successfully performed regex-based product searches and generated the matching report.

Q3. University Registration Validation

Aim

To validate student registration details using Regular Expressions.

Algorithm

15. Start the program.
16. Read all student details.
17. Validate each field using regex.
18. Display validation messages.
19. Generate registration status.
20. Stop.

Code

```
import re
reg=input("Enter Register Number: ")
email=input("Enter Institutional Email: ")
course=input("Enter Course Code: ")
semester=input("Enter Semester: ")
mobile=input("Enter Mobile Number: ")
reg_valid=bool(re.fullmatch(r"\d{2}[A-Z]{3}\d{4}",reg))
email_valid=bool(re.fullmatch(r"[A-Za-z0-9._%+-]+@saveetha\.com",email))course_valid=bool(re.fullmatch(r"[A-Z]{3}\d{3}",course))
semester_valid=bool(re.fullmatch(r"[1-8]",semester))
mobile_valid=bool(re.fullmatch(r"[6-9]\d{9}",mobile))
print("Validation Results")
print("Register Number:","Valid" if reg_valid else "Invalid")
print("Email:","Valid" if email_valid else "Invalid")
print("Course Code:","Valid" if course_valid else "Invalid")
print("Semester:","Valid" if semester_valid else "Invalid")
print("Mobile:","Valid" if mobile_valid else "Invalid")
if all([reg_valid,email_valid,course_valid,semester_valid,mobile_valid]):
print("Registration Successful")
else:
print("Registration Failed")
```

Sample Input

23CSE1001

sumanth@saveetha.com

DSA301

3

9876543210

Sample Output

Validation Results

Register Number: Valid

Email: Valid

Course Code: Valid

Semester: Valid

Mobile: Valid

Registration Successful

Result

The program successfully validated the student details and generated the registration status.